# GRAPH

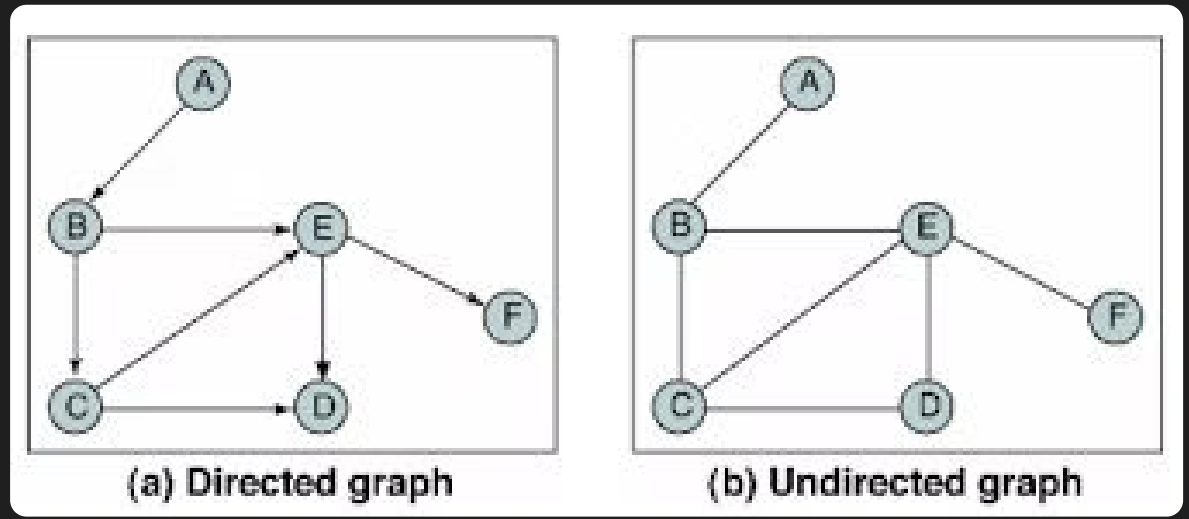
자료구조 스터디 Week13

학생회

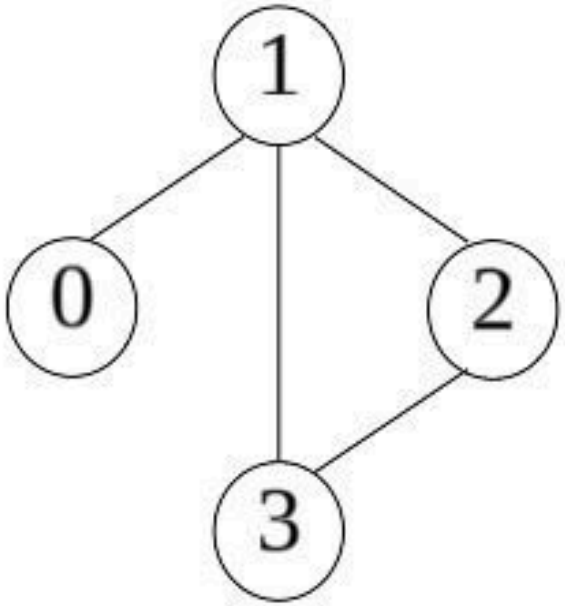
# INDEX

# 그래프

그래프는 정점(vertex)과 간선(edge)으로 이루어진 자료구조입니다. 간선의 종류에 따라 무향그래프, 유향그래프 등으로 구분할 수 있습니다.

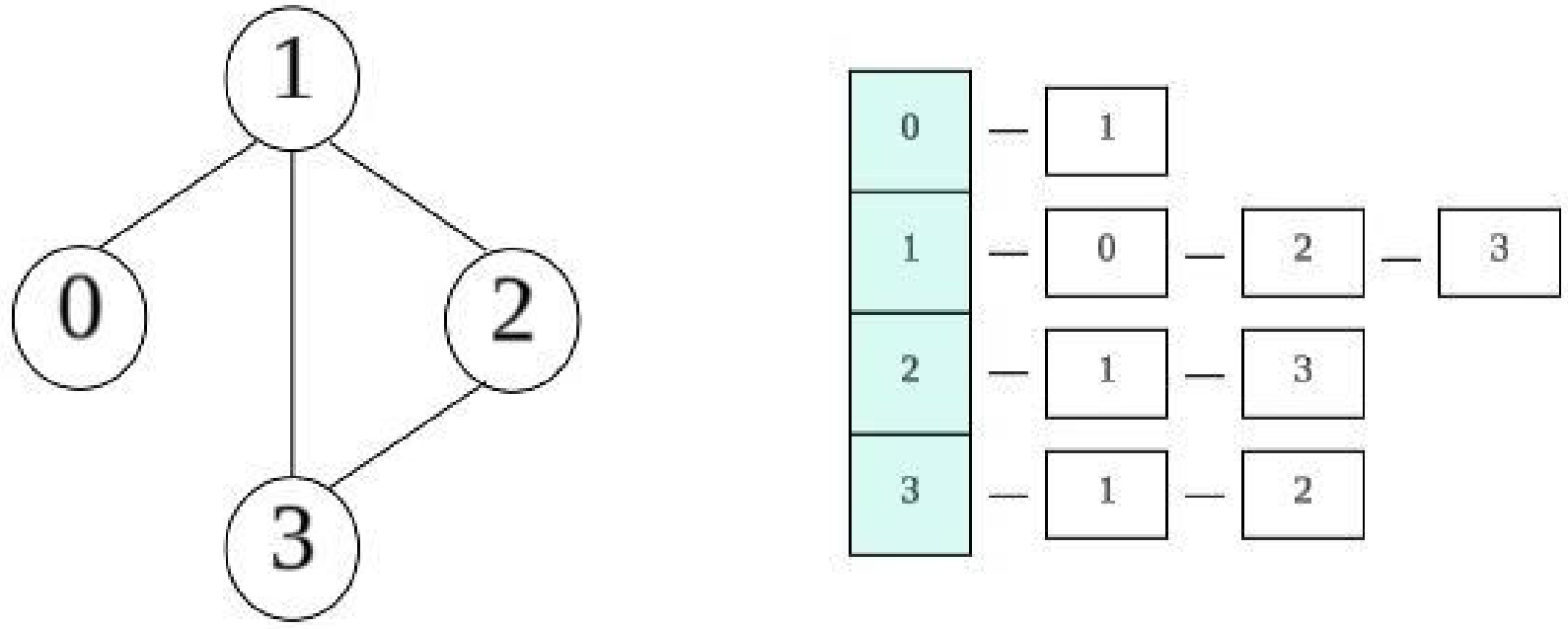


# 인접 행렬 표현법




| | 0 | 1 | 2 | 3 |
|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 0 |
| 1 | 1 | 0 | 1 | 1 |
| 2 | 0 | 1 | 0 | 1 |
| 3 | 0 | 1 | 1 | 0 |

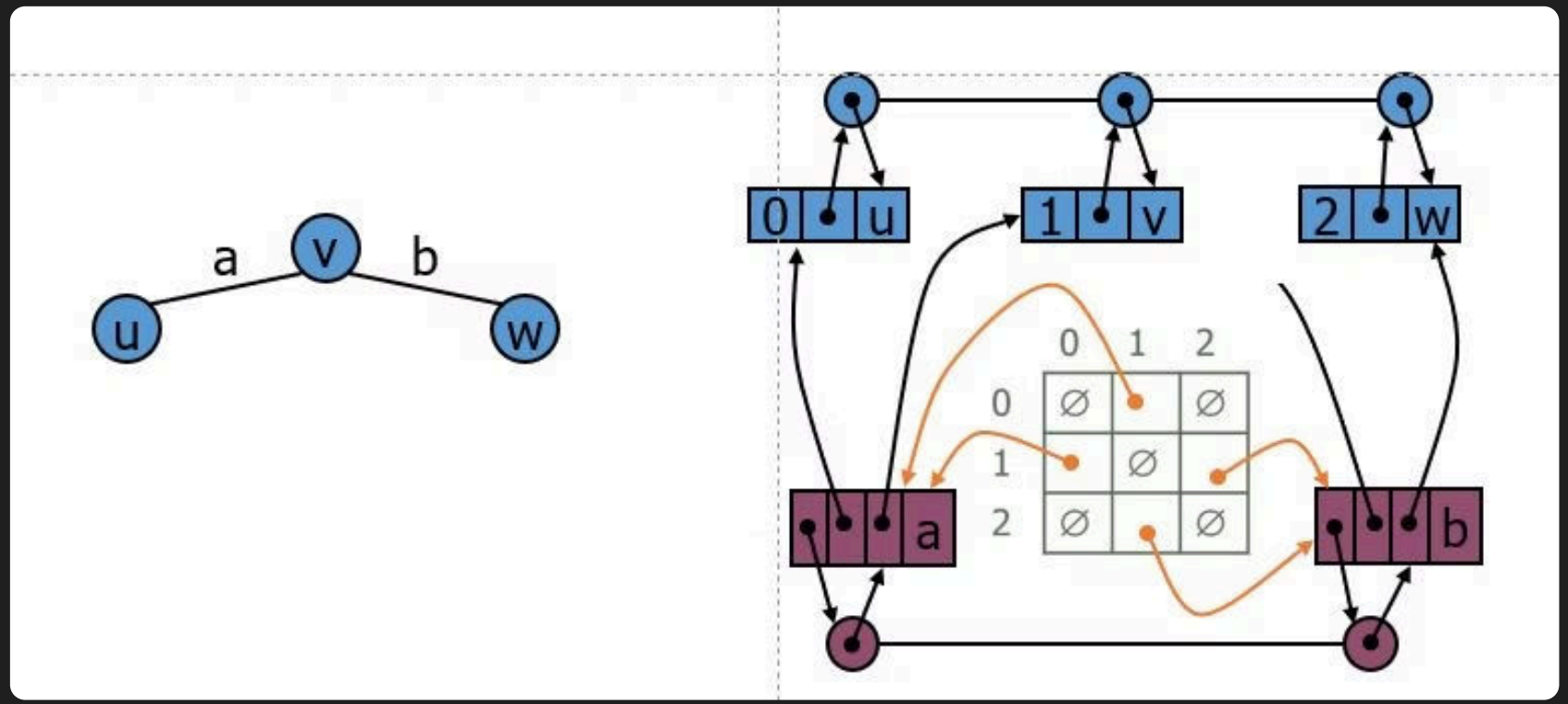
# 인접 리스트 표현법



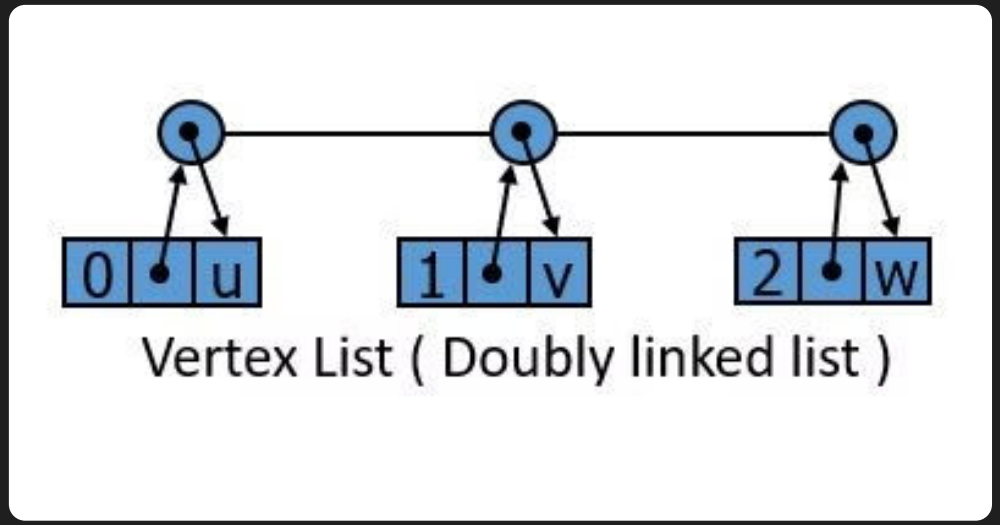
# ADJACENCY MATRIX REPRESENTATION

# 인접 행렬 표현법

행렬의 각 셀은 두 정점  u와 v를 잇는 간선이 존재하면 해당 간선의 주소를 저장

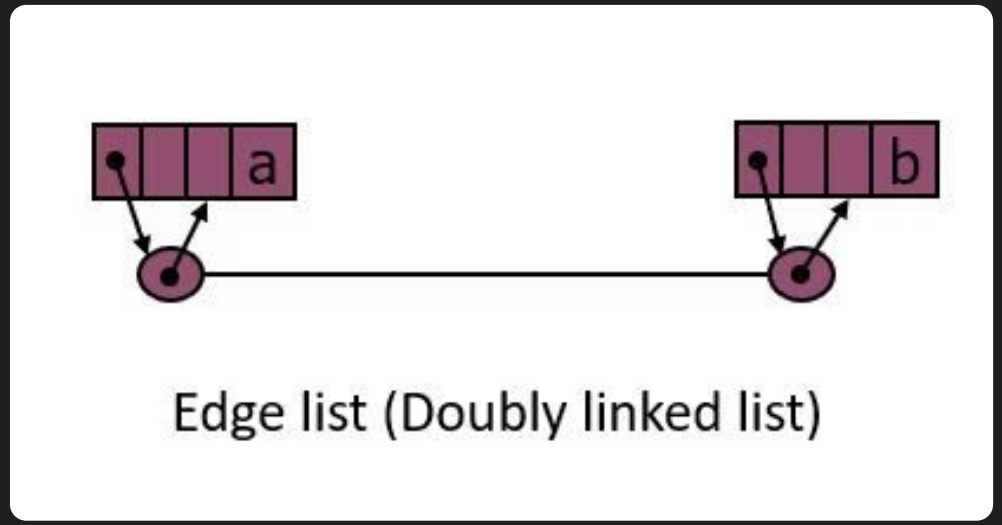


# 인접 행렬 표현법




## 정점 구조체

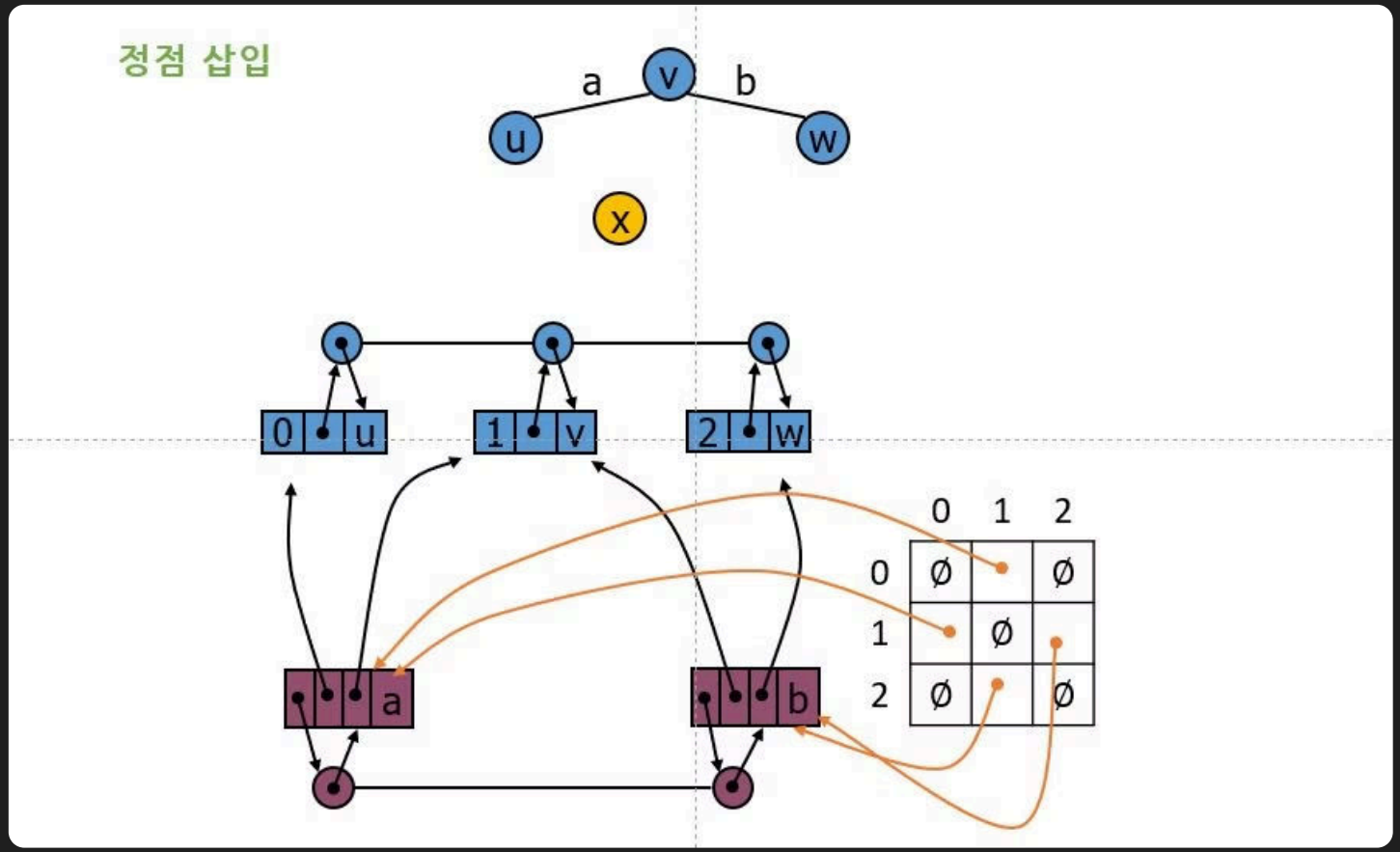
vertex_id, matrix_id, next, prev



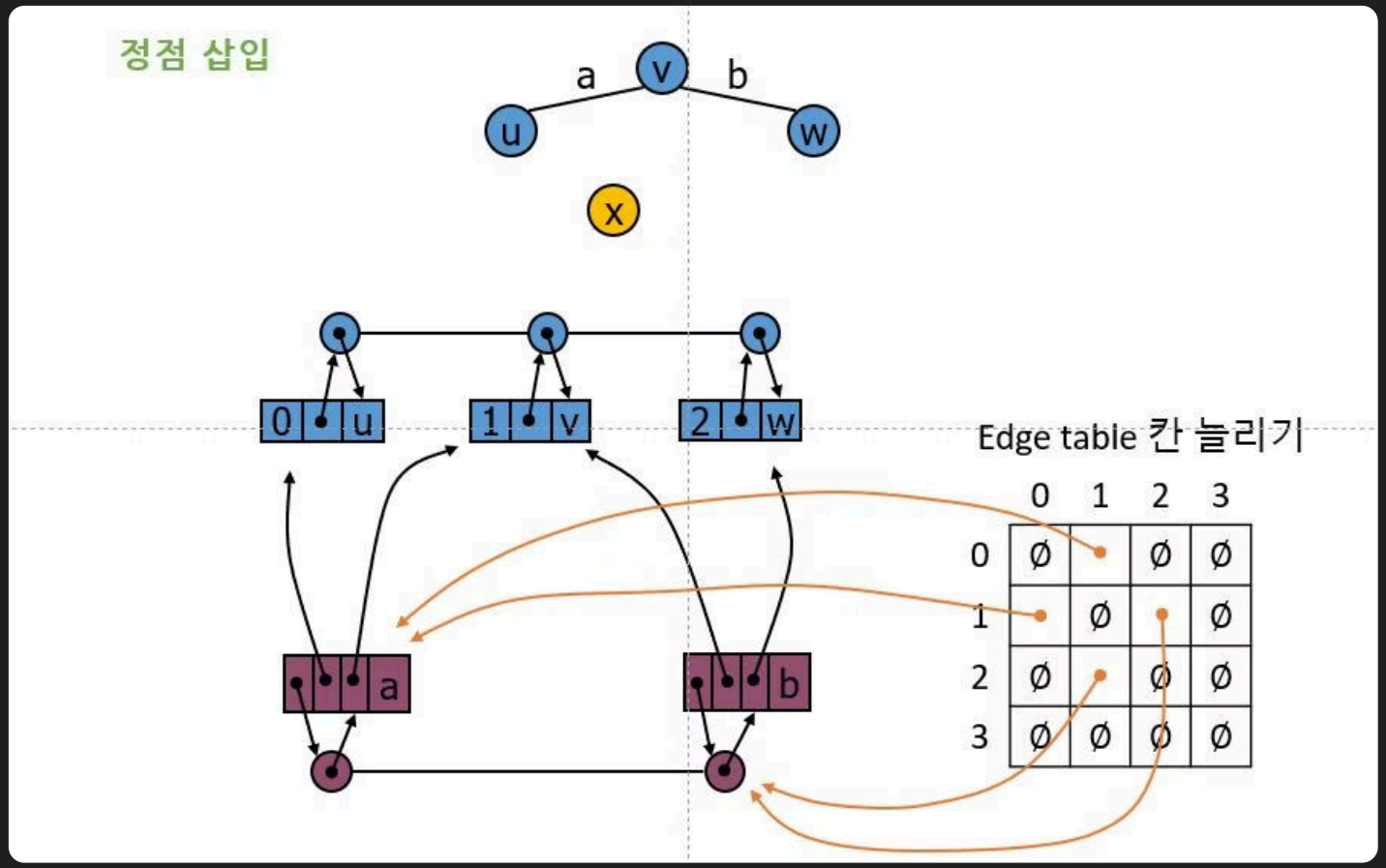

## 간선 구조체

src, dst, next, prev

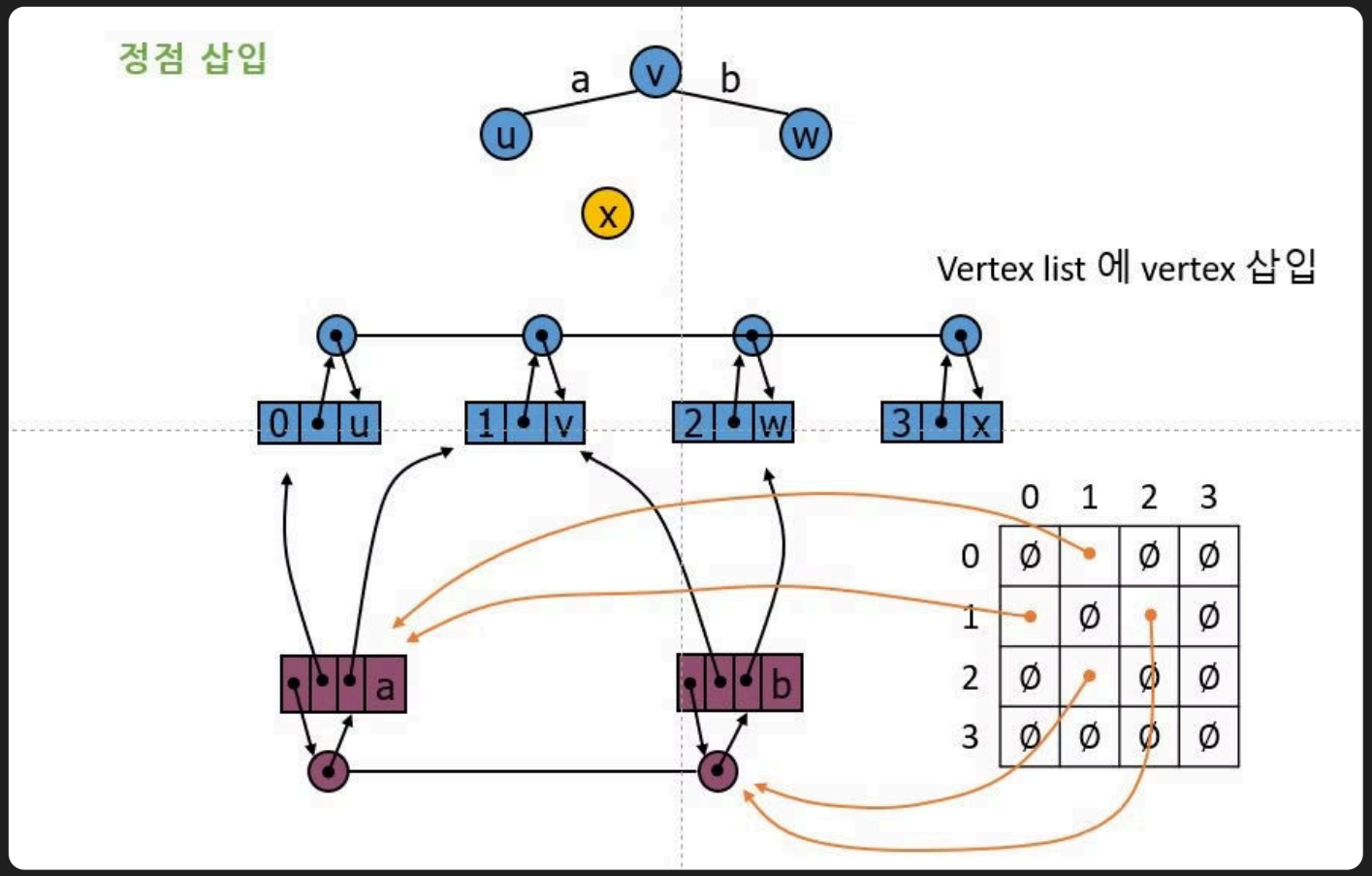
# 정점 삽입



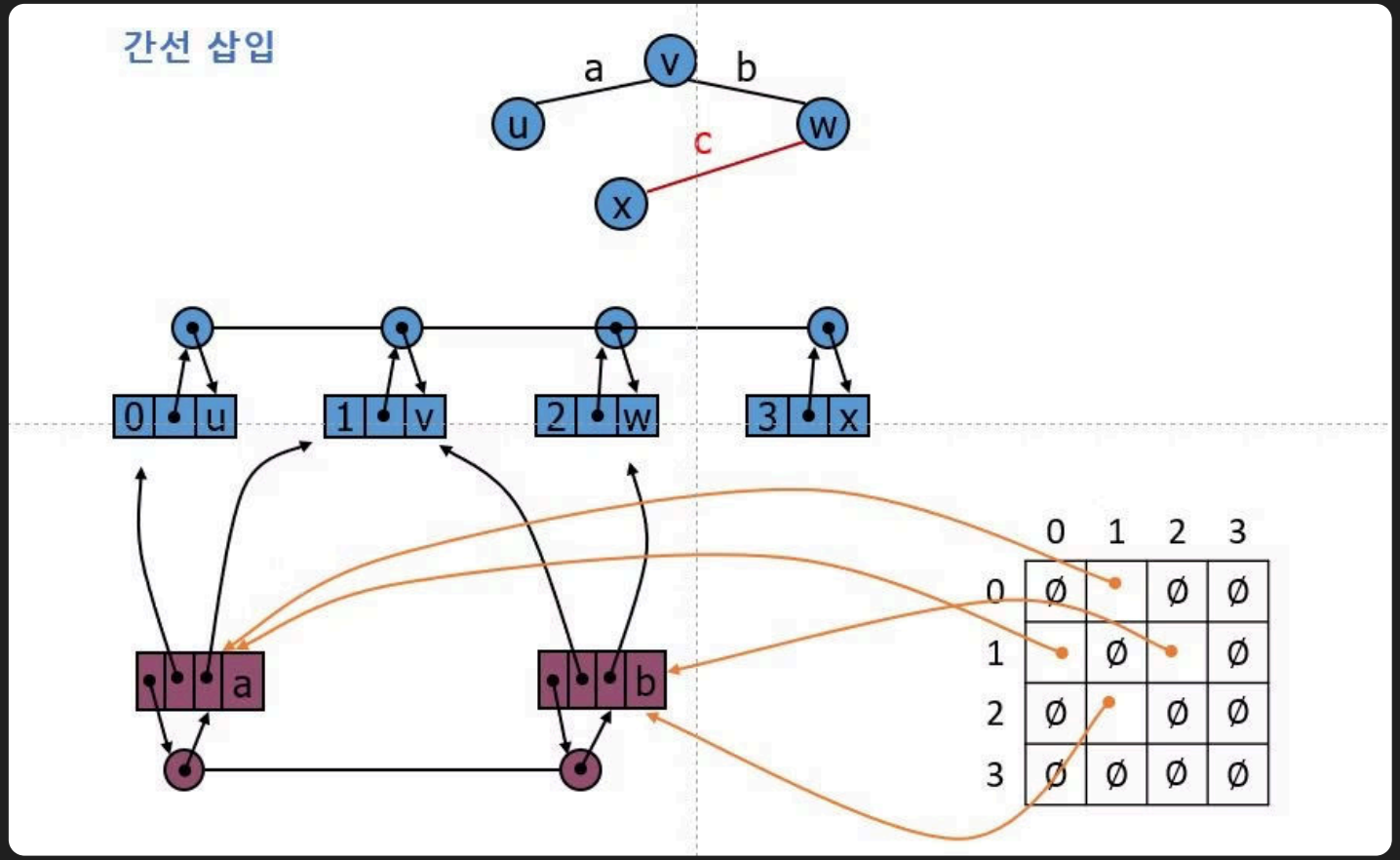
# 정점 삽입



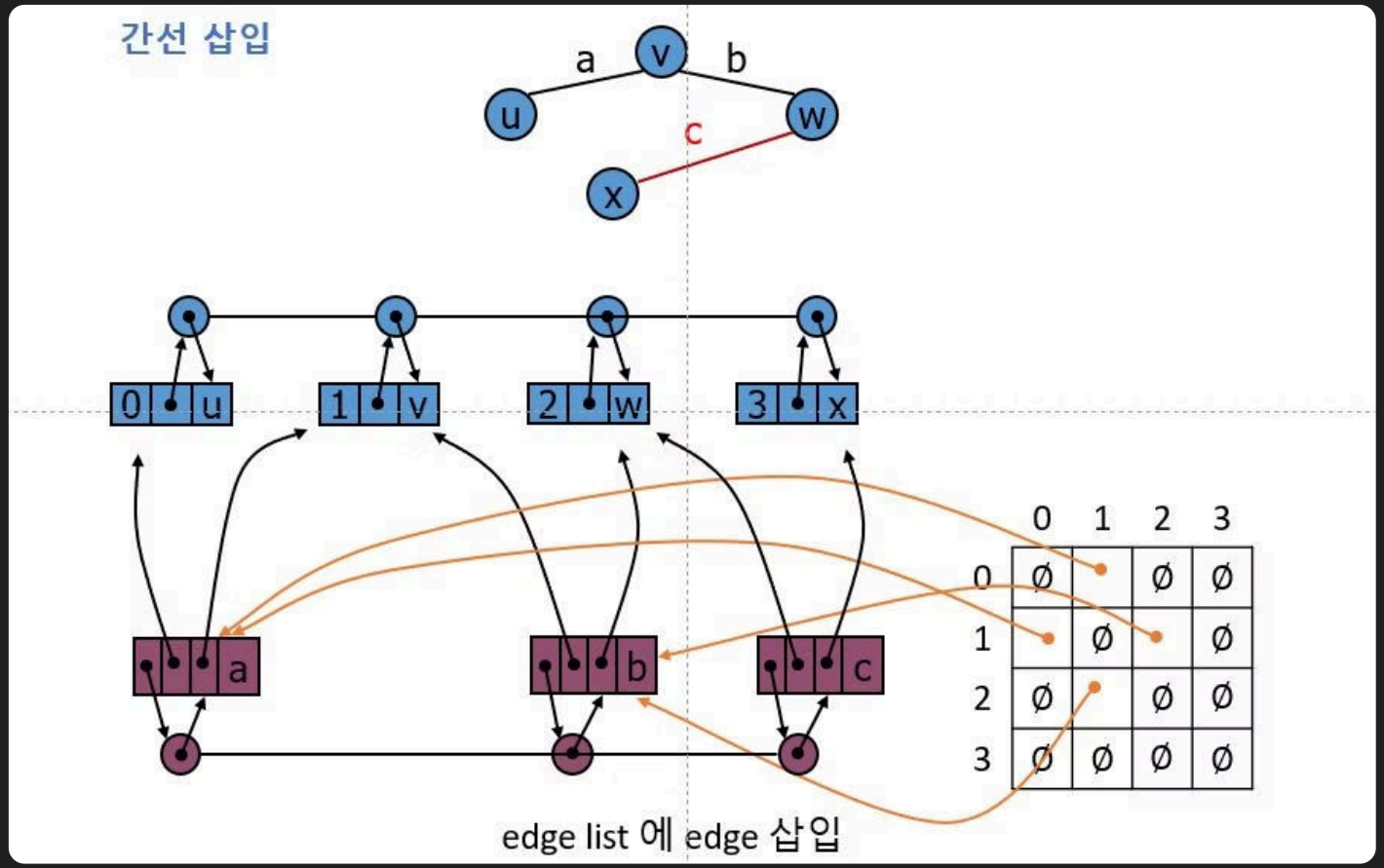
# 정점 삽입



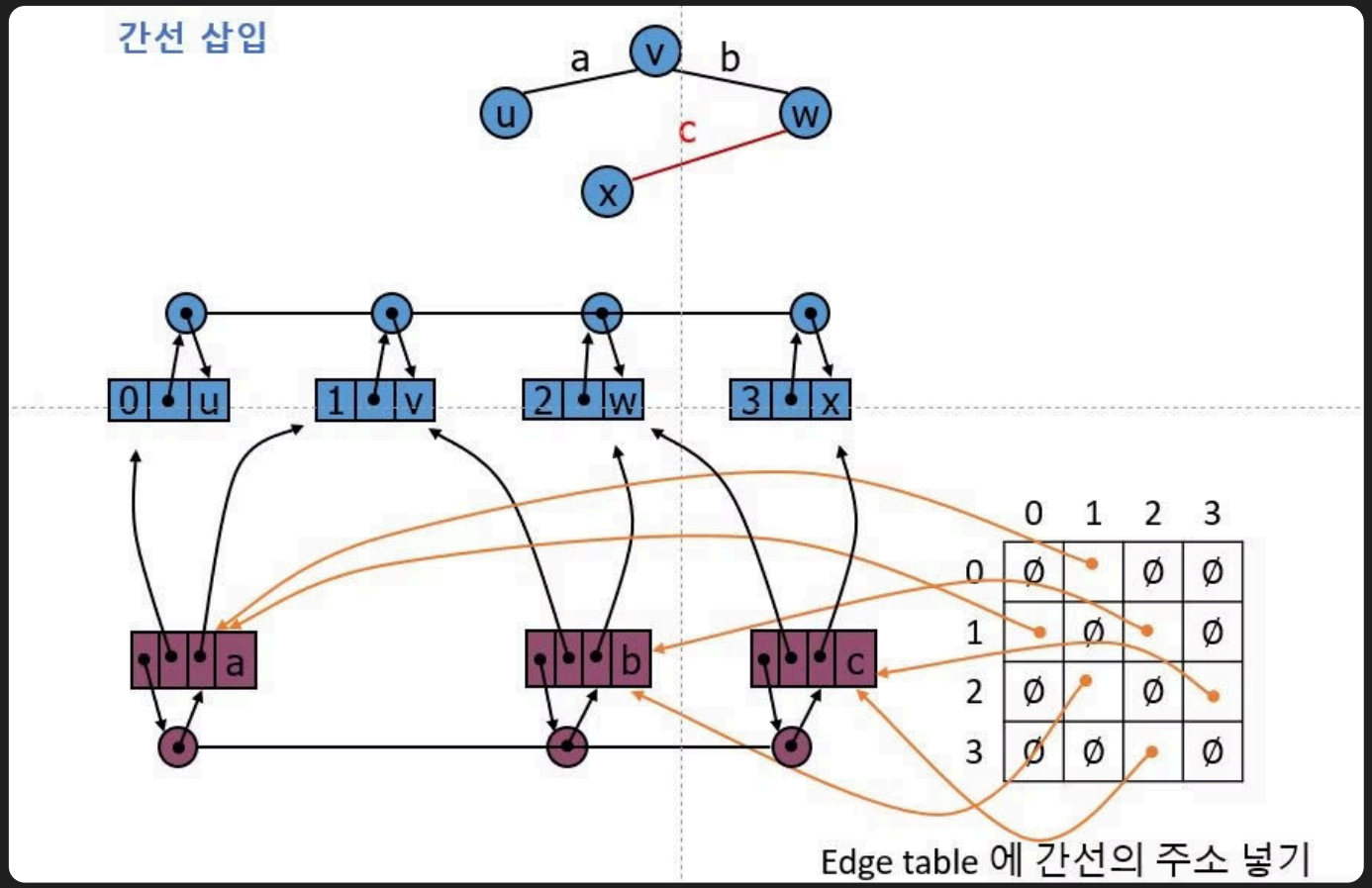
# 간선 삽입



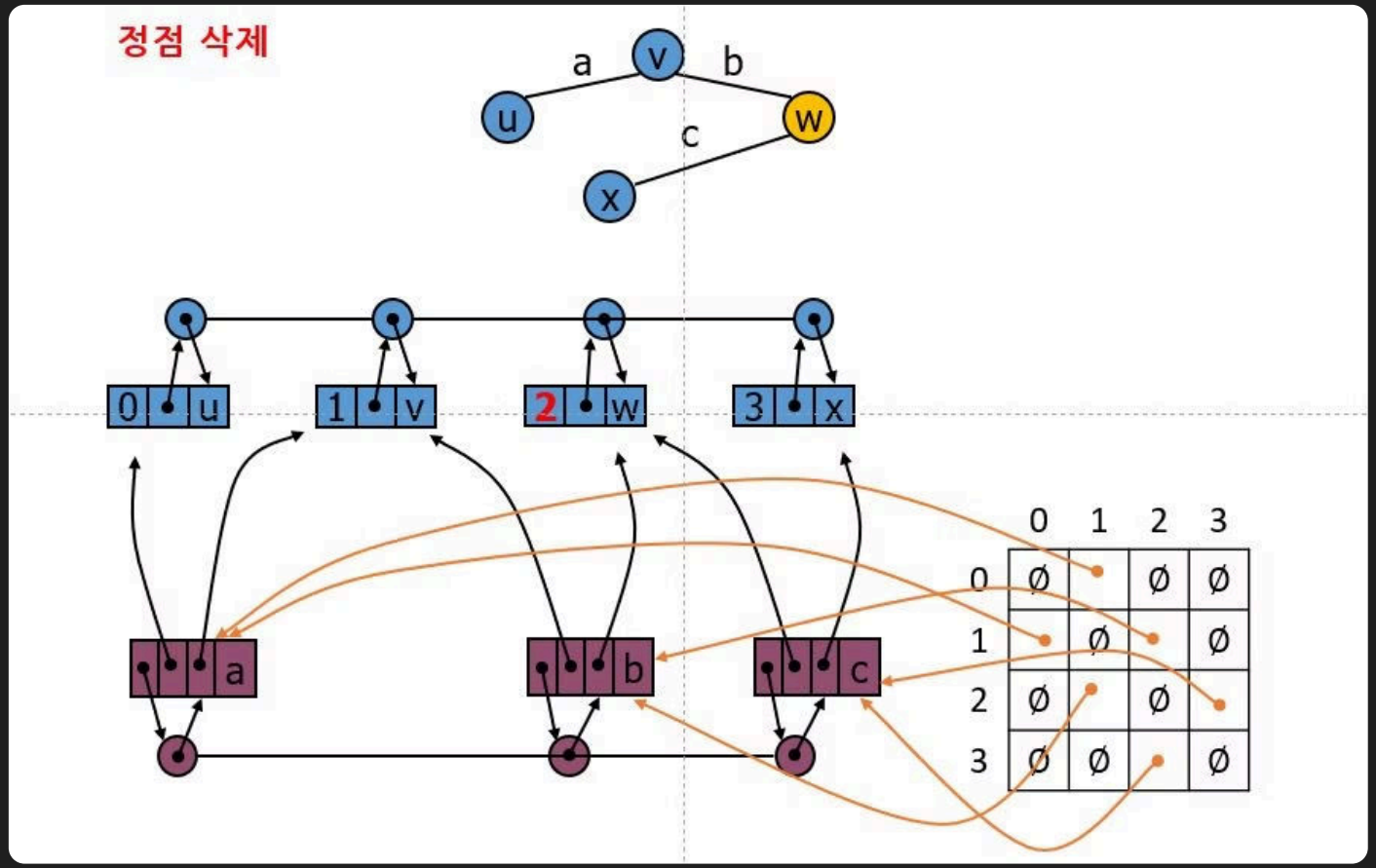
# 간선 삽입



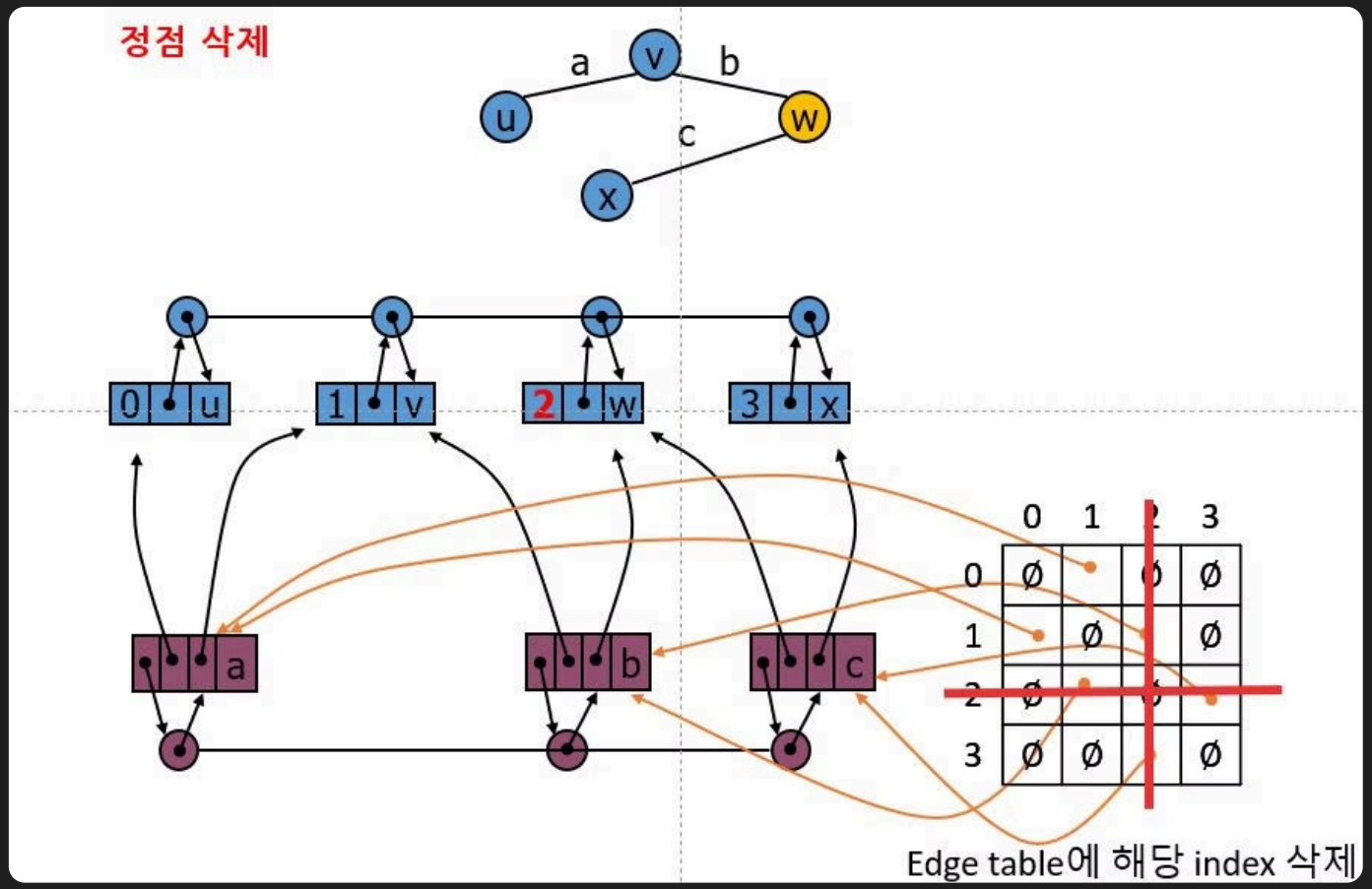
# 간선 삽입



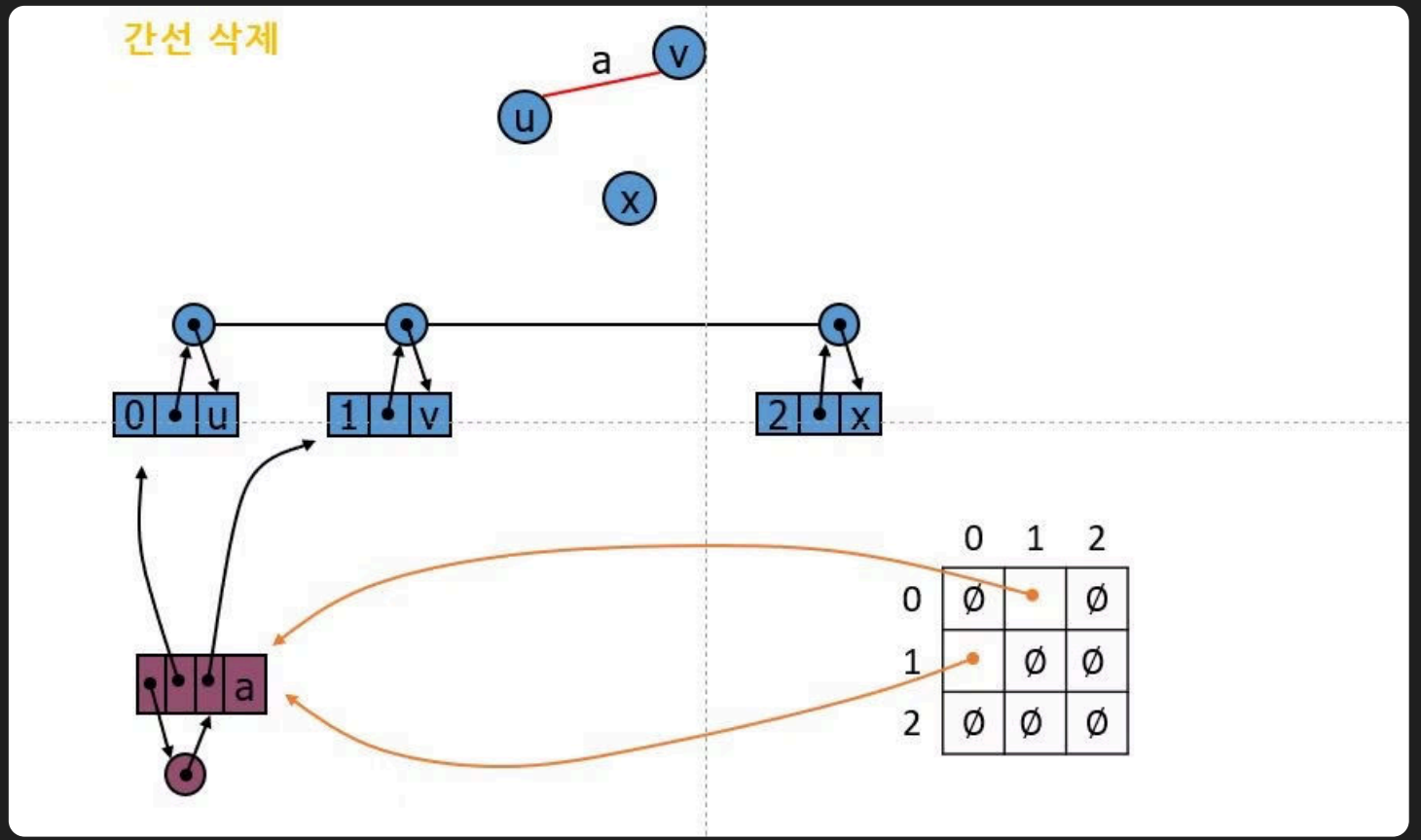
# 정점 삭제



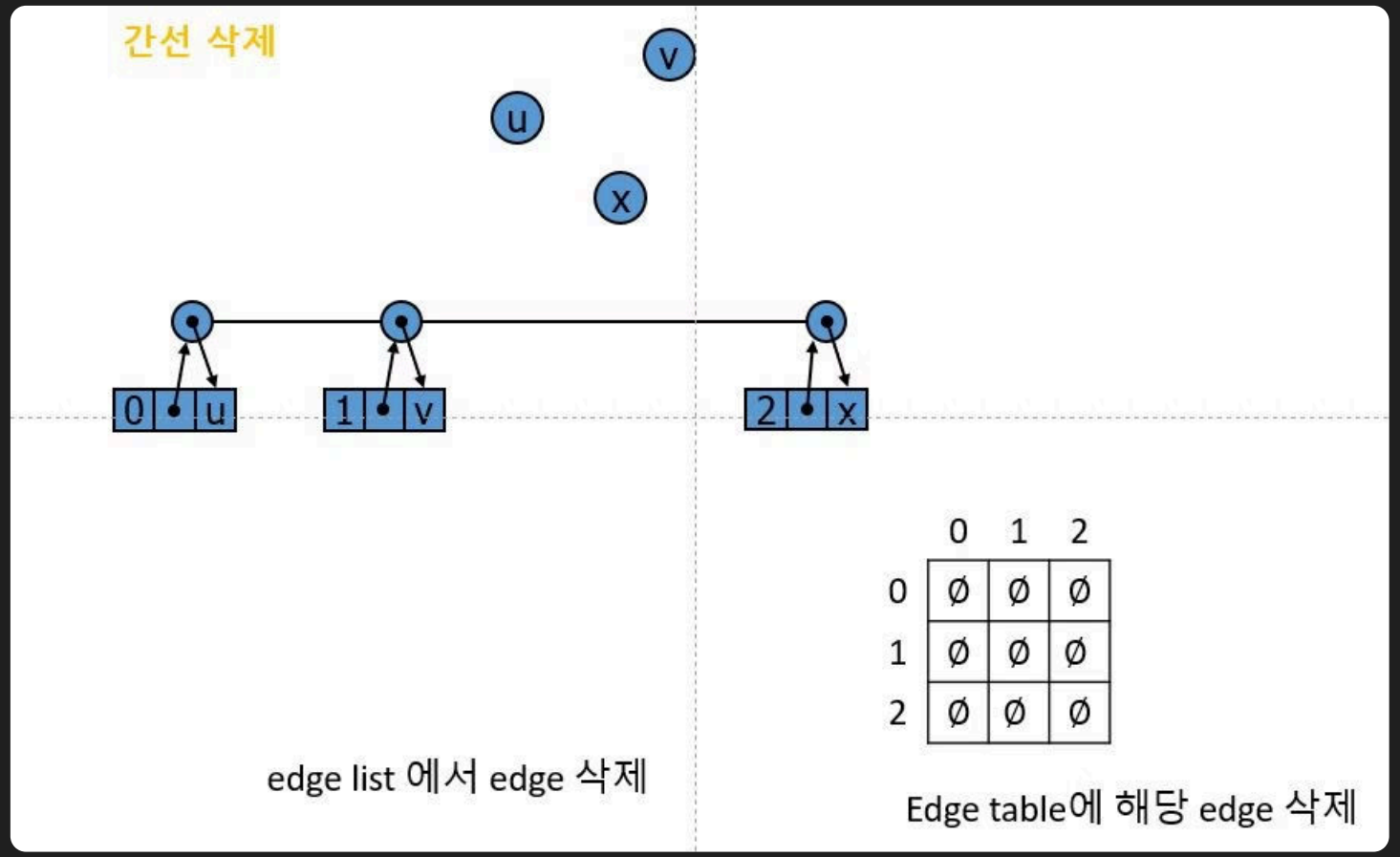
# 정점 삭제



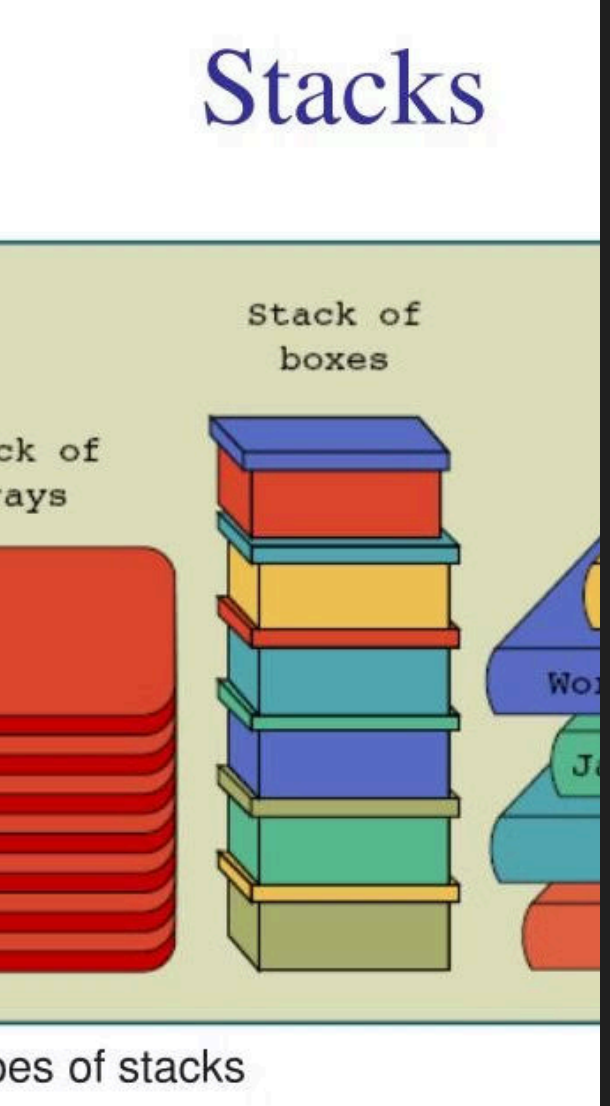
# 간선 삭제



# 간선 삭제



# IMPLEMENTATION




# 그래프의 구현 방법

1

## Adjacency Matrix Representation

인접 행렬 표현법

2

## Array Of Adjacency List Representation

인접 리스트 표현법

# 그래프의 기본 연산

1 **InsertVertex**

정점 삽입

2 **EraseVertex**

정점 삭제

3 **InsertEdge**

간선 삽입

4 **EraseEdge**

간선 삭제

5 **FindVertex**

vertexId로 정점 탐색

6 **Print_adjacent**

해당 정점에 인접한 정점 출력

# ADJACENCY MATRIX REPRESENTATION

# 정점 구조체

```
5   struct Vertex {
6           int vertexId;
7           int matrixId;
8           Vertex* prev;
9           Vertex* next;
10
11          Vertex() {
12                  vertexId = matrixId = -1;
13                  prev = next = nullptr;
14          }
15
16          Vertex(int vertexId) {
17                  this->vertexId = vertexId;
18                  matrixId = -1;
19                  prev = next = nullptr;
20          }
21  };
```

# 간선 구조체

```
23  struct Edge {
24          Vertex* src;
25          Vertex* dst;
26          Edge* prev;
27          Edge* next;
28
29          Edge() {
30                  src = dst = nullptr;
31                  prev = next = nullptr;
32          }
33
34          Edge(Vertex* src, Vertex* dst) {
35                  this->src = src;
36                  this->dst = dst;
37                  prev = next = nullptr;
38          }
39  };
```

# 정점 리스트 - 멤버변수 및 생성자

```
41  class VertexList {
42  private:
43          Vertex* header;
44          Vertex* trailer;
45
46  public:
47          VertexList() {
48                  header = new Vertex();
49                  trailer = new Vertex();
50                  header->next = trailer;
51                  trailer->prev = header;
52                  header->matrixId = -1;
53          }
```

# 정점 리스트 - Insert, Erase, Find

```
55  void insertVertex(Vertex* newVertex) {    // push_back()
56          newVertex->prev = trailer->prev;
57          newVertex->next = trailer;
58          trailer->prev->next = newVertex;
59          trailer->prev = newVertex;
60          newVertex->matrixId = newVertex->prev->matrixId + 1;    // modified
61  }
62
63  void eraseVertex(Vertex* delVertex) {
64          for (Vertex* cur = delVertex->next; cur != trailer; cur = cur->next) {
65                  cur->matrixId--;
66          }
67
68          delVertex->prev->next = delVertex->next;
69          delVertex->next->prev = delVertex->prev;
70          delete delVertex;
71  }
72
73  Vertex* findVertex(int vertexId) {
74          for (Vertex* cur = header->next; cur != trailer; cur = cur->next) {
75                  if (cur->vertexId == vertexId)
76                          return cur;
77          }
78
79          return nullptr;
80  }
```

# 간선 리스트 - 멤버변수 및 생성자

```
92  class EdgeList {
93  private:
94          Edge* header;
95          Edge* trailer;
96
97  public:
98          EdgeList() {
99                  header = new Edge();
100                 trailer = new Edge();
101                 header->next = trailer;
102                 trailer->prev = header;
103         }
```

# 간선 리스트 - Insert, Erase

```
105         void insertEdge(Edge* newEdge) {    // push_back()
106             newEdge->prev = trailer->prev;
107             newEdge->next = trailer;
108             trailer->prev->next = newEdge;
109             trailer->prev = newEdge;
110         }
111
112         void eraseEdge(Edge* delEdge) {
113             delEdge->prev->next = delEdge->next;
114             delEdge->next->prev = delEdge->prev;
115             delete delEdge;
116         }
117     };
```

# 그래프 - 멤버변수

```
119   class Graph {
120   private:
121           vector<vector<Edge*>> EdgeMatrix;
122           VertexList vList;
123           EdgeList eList;
124
```

# 그래프 - InsertVertex

```
126     void insertVertex(int vertexId) {
127         if (vList.findVertex(vertexId) != nullptr) {
128             cout << "Exist\n";
129             return;
130         }
131 
132         Vertex* newVertex = new Vertex(vertexId);
133 
134         for (int i = 0; i < EdgeMatrix.size(); i++) {
135             EdgeMatrix[i].push_back(nullptr);
136         }
137         EdgeMatrix.push_back(vector<Edge*>(EdgeMatrix.size() + 1, nullptr));
138 
139         vList.insertVertex(newVertex);
140     }
```

# 그래프 - EraseVertex

```
142     void eraseVertex(int vertexId) {
143         Vertex* delVertex = vList.findVertex(vertexId);
144
145         if (delVertex == nullptr)
146             return;
147
148         int matrixId = delVertex->matrixId;
149
150         for (int i = 0; i < EdgeMatrix.size(); i++) {
151             if (i != matrixId) {
152                 if (EdgeMatrix[i][matrixId] != nullptr)
153                     eList.eraseEdge(EdgeMatrix[i][matrixId]);
154
155                 EdgeMatrix[i].erase(EdgeMatrix[i].begin() + matrixId);
156             }
157         }
158
159         EdgeMatrix.erase(EdgeMatrix.begin() + matrixId);
160         vList.eraseVertex(delVertex);
161     }
```

# 그래프 - InsertEdge

```
163         void insertEdge(int srcVertexId, int dstVertexId) {
164             Vertex* src = vList.findVertex(srcVertexId);
165             Vertex* dst = vList.findVertex(dstVertexId);
166 
167             int srcMatrixId = src->matrixId;
168             int dstMatrixId = dst->matrixId;
169 
170             if (EdgeMatrix[srcMatrixId][dstMatrixId] != nullptr ||
171                 EdgeMatrix[dstMatrixId][srcMatrixId] != nullptr) {
172                 cout << "Exist\n";
173                 return;
174             }
175 
176             Edge* newEdge = new Edge(src, dst);
177             eList.insertEdge(newEdge);
178             EdgeMatrix[srcMatrixId][dstMatrixId] = newEdge;
179             EdgeMatrix[dstMatrixId][srcMatrixId] = newEdge;
180         }
```

# 그래프 - EraseEdge

```
182     void eraseEdge(int srcVertexId, int dstVertexId) {
183         Vertex* src = vList.findVertex(srcVertexId);
184         Vertex* dst = vList.findVertex(dstVertexId);
185 
186         int srcMatrixId = src->matrixId;
187         int dstMatrixId = dst->matrixId;
188 
189         if (EdgeMatrix[srcMatrixId][dstMatrixId] == nullptr ||
190             EdgeMatrix[dstMatrixId][srcMatrixId] == nullptr) {
191             cout << "None\n";
192             return;
193         }
194 
195         Edge* delEdge = EdgeMatrix[srcMatrixId][dstMatrixId];
196         eList.eraseEdge(delEdge);    // modified
197         EdgeMatrix[srcMatrixId][dstMatrixId] = nullptr;
198         EdgeMatrix[dstMatrixId][srcMatrixId] = nullptr;
199     }
```

# 그래프 - Print_adjacent

```
201         void print_Adjacent(int vertexId) {
202                 Vertex* curVertex = vList.findVertex(vertexId);
203                 int matrixId = curVertex->matrixId;
204
205                 for (int i = 0; i < EdgeMatrix[matrixId].size(); i++) {
206                         if (EdgeMatrix[matrixId][i] != nullptr)
207                                 cout << vList.findVertex_by_matrixId(i)->vertexId << ' ';
208                 }
209         }
210
```

# PROBLEM SOLVING

# THANK'S FOR WATCHING